CUDA Research Notes

Hardware, execution model, scheduling, toolchain and API

Condensed summary of five research notes

HADJI RAYAN

Contents

1 The Physical GPU 1

2 The CUDA Execution Hierarchy 2

3 The Warp Scheduler (the Main Thing) 3

4 CUDA, CUDA Architecture, CUDA C, CUDA Toolkit 3

5 CUDA Functions and Syntax 5

1. The Physical GPU

Hierarchy: a GPU chip contains several **GPCs**, each containing several **SMs**. One SM contains **4 sub-partitions**; each sub-partition has **1 warp scheduler and 32 CUDA cores**.

Component	What it is
SM (Streaming Multiprocessor)	The GPU's rough equivalent of a CPU core, but not the same thing. One SM holds ∼128 execution lanes, 4 warp schedulers, its own registers, its own shared memory and its own L1 cache.
CUDA core (FP32 lane)	One narrow arithmetic unit: it does $a+b$, $a\times b$ or a comparison on floating-point numbers; CUDA cores also do INT32 and bitwise operations. 128 per SM. Very "dumb" next to a CPU core: no branch prediction, no out-of-order execution, no private cache. Just a lane.
Warp scheduler	Small control unit, 4 per SM. Every clock cycle each scheduler picks *one ready warp* and issues *one instruction* to it.
Sub-partition (processing block)	1 scheduler + 32 CUDA cores. $4\times 32 = 128$ cores per SM, and 32 lanes = exactly one warp's width.

Memory and its cost

Memory	Location	Size	Latency	Visible to
Registers	inside SM	65,536 × 32-bit / SM	∼1 cycle	one thread
Shared memory (`__shared__`)	inside SM	∼100–230 KB / SM	∼20–30 cycles	all threads of one block
L1 cache	inside SM	shares memory with shared	∼30 cycles	automatic
L2 cache	on chip	tens of MB	∼200 cycles	all SMs
VRAM (global)	off-chip	8–24 GB	∼400–800 cycles	everything

The key number is 400–800 cycles. One VRAM read costs about as much as ∼600 arithmetic operations. A GPU that just waited for memory would be useless: everything about warps and schedulers exists to *hide* this latency.

Flow: CUDA program → GPU → SM → warp scheduler → CUDA cores → calculation.

Notes. Kernel = function, and 1 lane = 1 CUDA core, but a thread is not a core: a thread occupies one lane in the cycle it issues, and the lane is a CUDA core (needed later). Numbers such as "max 48 warps" are not fixed: they depend on the GPU, and some figures in these notes come from my own GPU, and others come from other GPUs, so the numbers here are just to understand: they are not 100% the ones we are going to use, nor 100% the ones we are not going to use; it just depends on the GPU we are going to use. Fixed facts are the structure above; max threads and max warps are computed per GPU.

2. The CUDA Execution Hierarchy

You write *one function* (a kernel) and run it across millions of data items. CUDA gives four nested levels, each mirroring a hardware scale: one lane (thread), one 32-wide instruction slot (warp), one SM with a shared scratchpad (block), the whole chip (grid).

Level 1 — Thread

One lane executing the kernel once, for one piece of data (one thread = one entity = one row of the data array). Each thread knows its index in its block, its block's index in the grid, the threads per block and number of blocks (both chosen by you), and has its own private registers.

Level 2 — Warp (the hardware unit)

32 consecutive threads executing in lockstep. You never create warps: you create blocks and threads and the hardware groups them.

- Exactly 32 threads, always, on every NVIDIA GPU; one instruction for all 32 lanes at once (**SIMT**, Single Instruction Multiple Threads).
- Cannot be split: if 5 lanes want to stop, the other 27 continue and those 5 idle.
- It is the unit of scheduling: the scheduler picks warps, not blocks or threads.
- It exists because the silicon is physically 32 lanes wide: one instruction gives 32 results in the time a CPU gets one (the ∼30× throughput advantage).

Divergence. If lanes of a warp disagree on a branch, the scheduler cannot issue two instructions at once, so it runs both branches serially, masking the inactive lanes each time. The warp takes twice as long.

```
if (i % 2 == 0)  a[i] = 1;   // 16 lanes active
else             a[i] = 2;   // other 16 lanes active
```

Level 3 — Block (CTA)

Up to **1024 threads**, guaranteed to run on **one SM**, able to share memory and synchronise. (Called a CTA in NVIDIA's formal docs.) Why group more than a warp?

1) **Shared memory:** all threads of a block share one fast scratchpad (`__shared__`); a warp alone has no memory of its own.
2) **Barriers:** `__syncthreads()` makes every thread in the block wait until all have arrived.

Rules: a block is placed whole on one SM and never spans two; blocks are fully independent (no shared memory, no synchronisation between blocks, any order, any number at a time). A block exists so the SM's many warps (up to 64 = 2048 threads) can cooperate on a tile of data. Without shared memory or barriers, block size is just an occupancy knob.

Sizing tip: use multiples of 32 (32/64/128/256/512/1024), otherwise lanes are wasted. A block is a group of warps. 128 or 256 is a good balance, neither too big nor too small.

Level 4 — Grid

All blocks launched by one kernel call, spread across all SMs. Can be 3D, usually 1D for entity arrays. The block count is limited only by grid-dimension limits, not by what fits on the GPU: you may launch 100,000 blocks, only dozens run at once and the rest queue. One launch = one grid = one launch overhead and one unit of asynchronous execution. It exists because one block is capped at 1024 threads on one SM, yet 10 million entities must be spread over ∼128 SMs; more blocks also let the scheduler fill the machine more evenly.

One sentence: I launch a grid of blocks; each block lands on one SM; the hardware chops each block into warps of 32; each warp scheduler issues one instruction at a time to 32 lanes.

3. The Warp Scheduler (the Main Thing)

An SM typically has **64 resident warps** (2048 threads), but only **4 instructions issue per cycle** (one per scheduler). Each scheduler therefore has ∼16 warps and each cycle asks: *which of my warps is ready?*

A warp is **not ready** if it is stalled on memory (∼500 cycles for VRAM), waiting on a dependency (needs the previous result), blocked at `__syncthreads()`, or its next instruction's pipe (e.g. FP32) is busy. The scheduler simply picks another ready warp. That is its whole job.

Why it is magic

A CPU on a memory stall tries to avoid it (caches, prefetching), reorders around it (out-of-order), or pays a costly context switch (thousands of cycles, state saved/restored). On a GPU there is **no context switch cost**: every resident warp's registers permanently sit in the register file, so switching from warp 3 to warp 11 is free (the scheduler just picks another index): zero-cost hardware multithreading.

So the answer to 500-cycle latency is not "avoid the stall" but "have 64 other warps ready": one warp waits ∼500 cycles while the other 63 keep the pipes busy (each warp issues about once per ∼4 cycles at best).

Latency is hidden by concurrency, not avoided by cleverness. This is the entire GPU performance model.

Occupancy

Occupancy = resident warps ÷ max warps per SM. With only 8 warps resident instead of 64, the scheduler runs out of candidates, latency becomes exposed and the SM idles.

Limiter	Effect
Registers	65,536 per SM. At 128 registers/thread only 512 threads (16 warps) fit → 25% occupancy.
Shared memory	If each block asks 100 KB, only 1–2 blocks fit per SM.
Block size	1024-thread blocks → max 2 per SM; small blocks → max 32 per SM (its own cap).
Max blocks / SM	∼32 on recent architectures.

You control: block size, registers per thread (`__launch_bounds__`, `-maxrregcount`) and shared memory per block. *Nsight Compute* reports occupancy and what limits it.

Rule of thumb: do not chase 100% blindly. 50% occupancy with clean coalesced access usually beats 100% with bad access patterns, but below ∼25% is usually a red flag.

Coalescing — the other half of performance

When a warp loads, its 32 lanes request addresses simultaneously and the memory system tries to merge them.

- **Coalesced:** 32 lanes read 32 consecutive floats (128 bytes) → only 1–4 memory transactions.
- **Strided:** each lane reads every 32nd float (lane 0 → [0], lane 1 → [32], . . .) → up to 32 transactions, about 8× slower.

This is exactly why **SoA** (structure of arrays) beats **AoS** (array of structures):

```
AoS: [x y z vx vy vz hp name sprite][x y z ...]...
     thread i reads pos.x -> stride = 88 bytes -> UNCOALESCED
SoA: pos.x: [x0][x1][x2][x3]...
     stride = 4 bytes -> PERFECTLY COALESCED
```

4. CUDA, CUDA Architecture, CUDA C, CUDA Toolkit

The pipeline, layer by layer

- **You write** CUDA C++ (`.cu`): standard C++ plus `__global__`, `__device__`, `«<»>`, `threadIdx`, `__shared__`, `__syncthreads()`. A language / source-level thing.
- **You compile** with `nvcc`, which splits the file: host code → your C++ compiler → x86 binary; device code → PTX → (driver JIT) → SASS.
- **You call** the **Runtime API** (`cudaMalloc`, `cudaMemcpy`, `cudaDeviceSynchronize`, `«<»>`) or the lower-level **Driver API** (`cuMemAlloc`, `cuLaunchKernel`); the runtime sits on the driver.
- **Installed:** the NVIDIA driver, shipped separately (you do not bundle it).
- **Runs on** GPU hardware (SMs, schedulers, cores, registers, shared memory, VRAM), characterised by its **compute capability**.

The terms

Term	What it is	Layer
CUDA	Umbrella brand: NVIDIA's parallel computing platform + programming model	brand
CUDA programming model	Host/device split, kernels, thread→warp→block→grid, memory spaces	abstraction
CUDA C / C++	C++ plus extensions; a dialect of C++, not a new language	language
`nvcc`	Compiler driver splitting host/device code (in the toolkit)	tool
CUDA Toolkit	The downloadable SDK: nvcc, headers, cudart, libraries, profilers, samples	toolchain
Runtime API	`cudaMalloc`, `cudaMemcpy`, `«<»>`: the friendly layer (in the toolkit)	software API
Driver API	`cuInit`, `cuMemAlloc`, `cuLaunchKernel`: low level (in the driver)	software API
PTX	Portable intermediate assembly that device code compiles to	IR
SASS	The actual machine instructions the GPU executes (from driver JIT or `cuobjdump`)	machine code
Compute Capability	Version number of what a GPU's hardware can do; a property of the GPU, not of CUDA	hardware spec
SM / scheduler / core / warp	Physical silicon and execution granularity	hardware
cuBLAS, cuFFT, Thrust, CUB, cuDNN	GPU libraries built on CUDA (in the toolkit)	libraries

"CUDA architecture" has three meanings

1) **The acronym:** Compute Unified Device Architecture = the programming model (host/device, kernels, thread hierarchy, memory spaces). Same thing as "CUDA".
2) **A GPU hardware generation:** "Ampere architecture", "Hopper", "Blackwell": the silicon design, identified by compute capability. Different from CUDA the platform.
3) **The software stack layering:** driver API / runtime API / libraries (occasionally "the CUDA software architecture").

Compute capability (MAJOR.MINOR: generation.refinement)

CC	Architecture	Example cards
7.5	Turing	GTX 1650, RTX 2060
8.0 / 8.6	Ampere	A100 / RTX 3060–3090
8.9	Ada Lovelace	RTX 4060–4090
9.0	Hopper	H100
10.x / 12.x	Blackwell	B200 / RTX 50-series

It determines max threads/SM, shared memory per block, registers, warp-level primitives, async copy, tensor cores and cluster support.

Gotcha: the installed CUDA version must be ≥ what your compiler targets and the driver must be new enough. CUDA 12.x code cannot run on a driver supporting only 11.4; you get `"no kernel image is available for execution on the device"`.

Summary: CUDA is the brand. The programming model is the abstraction (grids/blocks/warps/threads + memory spaces). CUDA C++ is the language you type, nvcc compiles it, the Toolkit is the box with nvcc, runtime, libraries and profilers. The driver executes your launch; the GPU hardware, described by its compute capability, actually runs the warps.

5. CUDA Functions and Syntax

Two worlds: **host code** (CPU, manages things) and **device code** (GPU, inside `__global__`). Classic beginner mix-up: `cudaMalloc` is host-side, `threadIdx` is device-side.

1. Host-side functions

Call	What it does	Simple version
`cudaMalloc(&d_ptr, bytes)`	reserves VRAM memory	"give me a box on the GPU"
`cudaFree(d_ptr)`	releases it	"throw the box away"
`cudaMemcpy(dst, src, bytes, kind)`	copies RAM ↔ VRAM	"move data across the bridge"
`cudaMemset(d_ptr, val, bytes)`	fills VRAM with a byte value	"zero it out"
`cudaDeviceSynchronize()`	waits for all GPU work to finish	"block until the GPU is done"
`cudaGetLastError()`	returns the last CUDA error	"did my launch work?"
`cudaGetDeviceProperties(&p, 0)`	reads the GPU's specs	"what card am I on?"
`cudaEventCreate/Record/Synchronize/ElapsedTime`	GPU-side timing	"stopwatch on the GPU clock"

The four `kind` values of `cudaMemcpy`:

- `cudaMemcpyHostToDevice`: RAM → VRAM (upload)
- `cudaMemcpyDeviceToHost`: VRAM → RAM (download)
- `cudaMemcpyDeviceToDevice`: VRAM → VRAM
- `cudaMemcpyDefault`: let CUDA figure it out

2. Device-side built-ins

Every thread gets these automatically: no declaration, no arguments. Each has `.x`, `.y`, `.z`; a 1D entity array only uses `.x`.

Variable	Type	Meaning	Changes per...
`threadIdx`	`uint3`	my index inside my block	thread
`blockIdx`	`uint3`	my block's index inside the grid	block
`blockDim`	`uint3`	threads per block	launch (constant)
`gridDim`	`uint3`	blocks in the grid	launch (constant)
`warpSize`	`int`	always 32	never

```
int i = blockIdx.x * blockDim.x + threadIdx.x;   // global thread index (1D)
```

Limits: `blockDim.x*blockDim.y*blockDim.z` ≤ 1024; `gridDim.x` $\leq 2^{31}-1$; `gridDim.y`, `gridDim.z` $\leq 65{,}535$; `warpSize` $= 32$ (fixed).

3. Function / memory qualifiers

Keyword	Runs on	Called from
`__global__`	GPU	CPU (with `<<<>>>`): this is your kernel
`__device__`	GPU	GPU (helper function called by a kernel)
`__host__`	CPU	CPU (default for everything; can combine: `__host__ __device__`)
`__shared__`	—	declares per-block shared memory

4. Launch syntax

```
kernel<<<blocksPerGrid, threadsPerBlock>>>(arg1, arg2);
kernel<<<dim3(bx, by), dim3(tx, ty)>>>(...);        // 2D / 3D version
```

This is not C++ but a CUDA extension: `nvcc` turns it into a driver call. It is *not* an ordinary function call: it **queues** work and returns immediately.